

- **Limits:** Set the maximum resources an LLM application can use. This prevents a single pod from consuming all the resources.
- Implement horizontal pod autoscaling to automatically scale the number of pods based on CPU or memory usage.
- Implement vertical pod autoscaling to adjust the CPU and memory requests for running pods based on actual usage
- **Resource quotas:** Set resource quotas to limit the overall resource usage in a namespace, ensuring fair resource allocation.
- **Node selectors and affinity:** Assign pods to specific nodes based on labels. Define rules for how pods should be placed on nodes, considering factors like co-location and anti-affinity.
- **Persistent storage:** Use persistent volumes (PV) and persistent volume claims (PVC) to manage storage needs.

Deploying to Kubernetes: Use kubectl to apply the Kubernetes configuration files and deploy LLM.

Monitor the deployment process to ensure everything is running smoothly.

- Implement network policies to secure LLM deployment.
- Isolate different parts of the Kubernetes cluster, ensuring that only authorised traffic can flow between pods.
- Encrypt all data transmitted between the LLM and other systems using protocols like TLS.
- Set Kubernetes RBAC (role-based access control), which ensures that only authorised personnel can access specific resources and perform certain actions.

Monitoring and scaling: Use monitoring tools like Prometheus and Grafana to keep track of resource usage and performance metrics. As part of monitoring:

- Adjust the number of pods and resources as needed based on the monitoring data to handle varying workloads.
- Set up backup and recovery processes to protect data and ensure quick recovery in case of failures.
- Regularly review access logs to detect any unusual or unauthorised activity.

Optimisation: Continuously optimise resource allocation and performance to ensure efficient operation of LLMs.

Example of LLM deployment

The following code snippets briefly explain how to deploy the GPT-4 language model on Kubernetes using a pre-built Docker image from Hugging Face.

Create a Docker image containing the GPT-4 model using Hugging Face's Docker file:

```
FROM huggingface/huggingface-transformers:latest
```

```
COPY requirements.txt /app/
```

```
RUN pip install -r /app/requirements.txt
```

```
COPY . /app/
WORKDIR /app
CMD ["python", "app.py"]
```

```
```[_{{{CITATION}}}2{How to deploy Hugging Face models with gopal:ai] (https://www.gopal.ai/blog/how-to-deploy-hugging-face-models-with-gopal-ai)cker images.
```

Build the Docker image using the Docker image command:

```
Sh
```

```
docker build -t my-gpt4-model .
```

```
```[_{{{CITATION}}}3{How to deploy Hugging Face models with gopal:ai] (https://www.gopal.ai/blog/how-to-deploy-hugging-face-models-with-gopal-ai)
```

Next, push the Docker image to a container registry like Docker Hub or Google Container Registry:

```
Sh
```

```
docker tag my-gpt4-model <beharLLM-registry>/<beharLLM-image>
```

```
docker push <beharLLM-registry>/<beharLLM-image>
```

```
```[_{{{CITATION}}}4{Deploying ViT on Kubernetes with TF Serving - Hugging Face](https://huggingface.co/blog/deploy-tfserving-kubernetes)
```

Now, create a Kubernetes deployment YAML file to deploy the Docker image:

```
Yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: gpt-4-deployment
```

```
spec:
 replicas: 1
 selector:
 matchLabels:
 app: gpt-4
```

```
template:
 metadata:
 labels:
 app: gpt-4
```